# WISEConv: Closing the Sparsity-to-Speedup Gap in Masked Convolution with Worklist-Driven Execution

ISCA 2027 Submission **#897** – Confidential Draft – Do NOT Distribute!!


## Abstract

Masked convolution promises lower latency by computing only positions an upstream policy marks active, yet existing paths keep either throughput (tile skipping) or useful-compute ratio (gather-scatter), never both, failing to turn sparsity into proportional speedup. Contrary to the expectation that scattered work breaks dense efficiency, we find that restricting a dense tiled pipeline to scattered positions largely preserves per-tile throughput: irregularity is confined to row-level indirection outside the reduction loop, leaving channel-contiguous access and weight-tile reuse identical to dense. We therefore present WISEConv, which drives the unchanged dense pipeline with an active-output worklist. The worklist brings two costs dense execution never pays. Construction cost stays fixed per layer shape as compute shrinks with the active ratio, so WISEConv builds the worklist in one mask-space pass and reuses it across compatible operators. The worklist length, visible only on the device, varies per frame and layer and shifts the best tile configuration, so WISEConv performs on-device cost-aware configuration switching, weighing each candidate's gain against its selection cost. Across six GPUs and four workloads, WISEConv achieves a 2.08× geometric-mean latency reduction over the fastest baseline per workload-platform pair and reduces energy by 60.8–82.5% versus dense execution on embedded GPUs.



## Keywords

Sparse convolution, dynamic spatial sparsity, GPU runtime, deep learning systems, efficient inference

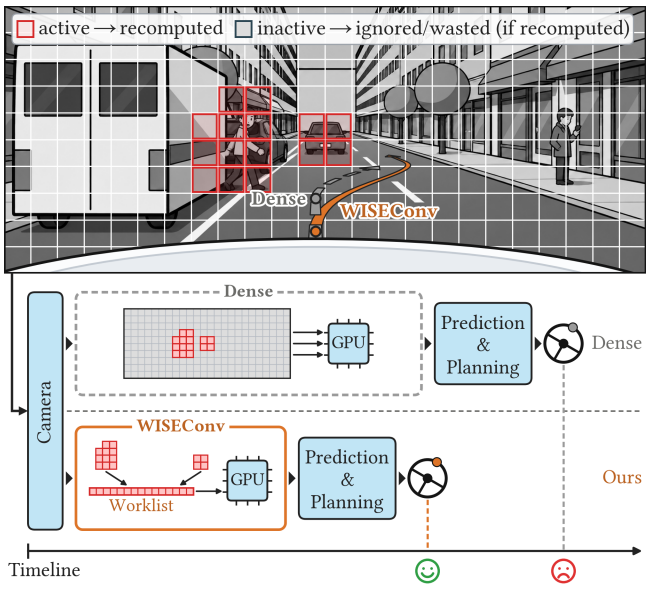




**Figure 1: Perception latency decides whether a vehicle can react in time. Only the active positions (red) of a frame need to be recomputed; dense convolution still computes all of them. WISEConv collects the active positions into a worklist and drives the dense tiled pipeline with it, which keeps the throughput of dense execution, so prediction and planning start earlier and the vehicle steers away sooner.**

## 1 Introduction

Dense convolution dominates per-frame latency in vision tasks where reaction time decides the outcome [3, 12, 47], such as a car avoiding a pedestrian stepping out of occlusion [26], a drone dodging an obstacle [13], or a robot returning a table-tennis ball [9] (Figure 1). Yet on a given input much of that computation is spatially redundant. A common remedy is masked convolution: an upstream policy marks spatially active positions on the dense feature grid, and a dedicated sparse convolution operator updates only these positions in the dense output. The mask may derive from event-camera increments [10, 28], temporal differences [20, 34], or learned spatial gates [25, 44]. These mask sources promise large reductions in convolution work, yet existing masked-convolution systems [10, 25, 34, 44] rarely translate the sparsity into proportional latency reduction.

Dense GPU convolution obtains high *throughput* (operations per second) from coordinate-driven, tiled execution. Each tile enumerates a contiguous block of output coordinates; those coordinates fix each output position's receptive field and write location, while the reduction over channels and kernel offsets follows a regular datapath with coalesced loads and weight-tile reuse. In masked convolution, reducing latency proportionally requires sustaining this throughput while achieving a high *useful-compute ratio* (the fraction of issued operations that produce needed outputs), but the per-frame position selection imposed by the mask conflicts with the contiguous coordinate assumption that sustains throughput, placing these two axes in tension.

Existing systems sacrifice one axis for the other. *Gather-scatter* [25, 44, 48] extracts exactly the active positions into a packed representation, computes, and scatters the results back, but is bottlenecked by the memory traffic of materializing their receptive fields. *Tile skipping* [10, 20, 34, 36] suppresses tiles that are entirely inactive and retains the dense pipeline, but wastes computation on partially active tiles. On an RTX 3080 running YOLOv8m [23] at batch 4 under temporal difference masks [34], 82.2% of convolution work is unnecessary. Gather-scatter sustains only 31.7% of the dense pipeline's throughput, reducing latency by just 43.8%; tile skipping reaches higher throughput but wastes 32.6% of issued operations, reducing latency by 36.9%, slower than gather-scatter despite its throughput advantage (§2.2).

We find that redirecting a dense tiled pipeline to compute only scattered active positions achieves both a high useful-compute ratio


*ISCA '27, Atlanta, Georgia, USA*
2027. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/XXXXXXX.XXXXXXX

and the throughput of dense execution. We collect these positions into a worklist and characterize (§3) the resulting tiled execution on a desktop and an embedded GPU, identifying why: because the input stays a dense grid, the irregularity reduces to one index per output row, outside the reduction loop that drives coalesced channel loads and weight-tile reuse, so per-tile throughput matches dense execution. When the tile configuration follows the worklist length, convolution time is affine in the active ratio with a slope matching the dense marginal cost; the residual gap is an intercept of 5.6–17.4% of dense time per layer (median per GPU and precision), dominated by kernel launch latency and fixed per-tile stalls such as barriers, not by throughput loss.

While this finding establishes that worklist-driven tiled execution can sustain both axes, closing the gap end to end requires solving two further challenges. First, *worklist construction cost*: deriving the worklist from the mask requires scanning the whole grid and costs time independent of the active count, so construction's share of latency grows as activity falls and accumulates when every layer rebuilds. Second, *worklist consumption efficiency*: the affine property holds for a given number of active positions under a matching tile configuration, but in deployment this number is visible only on the device after construction and shifts across frames and layers. Reading it back to the host stalls the per-frame path; adapting on the device avoids the stall but incurs a non-negligible cost per decision point (§3).

Based on these findings, we present WISEConv (**W**orklist-dr**I**ven ma**S**k**E**d **Conv**olution), which drives the dense tiled pipeline with an active-output worklist to achieve both a high useful-compute ratio and high throughput by co-designing worklist construction and consumption. To reduce construction cost, a single mask-space pass fuses output-mask propagation with per-tile compaction, operating on only mask and coordinate metadata; compatible operators reuse the worklist instead of reconstructing it. To sustain consumption efficiency, a persistent compute grid consumes the worklist without host synchronization, and a cost-aware selection mechanism adapts the tile configuration per frame, adding a candidate to a layer only where its expected gain exceeds the selection overhead.

This paper makes the following contributions:

- We characterize worklist-driven tiled execution, identifying why computing scattered active positions preserves dense-level throughput and where the remaining gap comes from. These findings motivate and constrain the design (§3).
- We present WISEConv, which co-designs worklist construction, worklist-driven tiled compute, and cost-aware tile selection to jointly sustain high useful-compute ratio and high throughput (§4).
- We evaluate WISEConv on four workloads across six GPUs. WISEConv achieves a 2.06× (FP32) and 2.10× (FP16) geometric-mean speedup over the fastest baseline per pair, and reduces energy by 60.8–82.5% relative to dense execution on embedded GPUs (§6).

## 2 Background and Motivation

### 2.1 Masked Convolution

Convolution runs inside the perception-action loop of many vision applications [3, 40, 45, 47, 51], where per-frame inference latency bounds system responsiveness [11–13, 21]. To cut this latency, masked convolution exploits dynamic spatial sparsity: an upstream policy emits a spatial mask, and the operator computes only the marked positions. The mask comes from sources including event-camera increments [10, 28], temporal residuals between video frames [4, 20, 33, 34, 46], or learned input-dependent gates [15, 25, 44, 48]. These sources differ in sensing modality and policy, but all expose the same dynamic spatial sparsity on a dense 2D feature grid, so a single masked-convolution runtime serves them all.

Masked convolution retains dense tensor layouts and arithmetic, restricting only which output coordinates are computed (Figure 2). It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$, a weight tensor $\mathbf{f} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ of kernel size $k$, and an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \ldots, H_{out} W_{out}\}$, where each coordinate $p \in \mathcal{G}$ indexes a spatial position $(h, w)$ and has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0,1\}^{H_{out} \times W_{out}}$. Spatial gating predicts $M_{out}$ directly; event and temporal policies instead supply an input mask $M_{in} \in \{0,1\}^{H_{in} \times W_{in}}$ whose active positions propagate their influence along the receptive field, so $p$ stays active whenever any input in $\mathcal{N}(p)$ is active,

$$M_{out}[p] = \begin{cases} 1, & \exists\, r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

$M_{out}$ thus identifies the output positions that the execution path is required to compute, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{f}$.

An execution path issues computation over a sequence of position slots $C$, where each slot is drawn from $\mathcal{G} \cup \{\perp\}$; $\perp$ denotes a padding slot that carries no output coordinate. The path must satisfy the *coverage contract*: every active coordinate is included, $\{p : M_{out}[p] = 1\} \subseteq \{c \in C : c \neq \perp\}$. For each non-$\perp$ slot $p \in C$ it

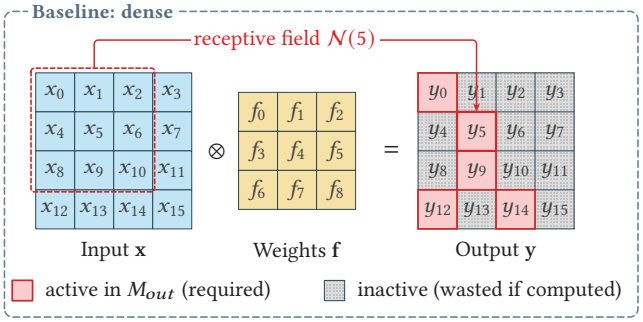


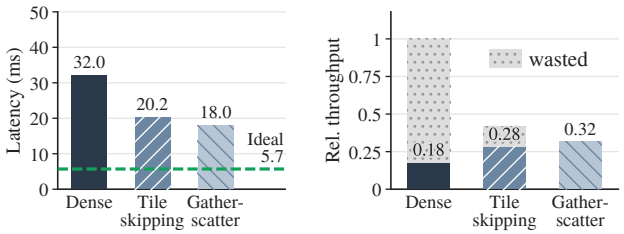

**Figure 2: Masked convolution and its dense execution. Each output $y_p$ aggregates its receptive field $\mathcal{N}(p)$ in $\mathbf{x}$ with the weights $\mathbf{f}$. Dense convolution computes every output in tiles of contiguous positions, which keeps loads coalesced and each weight tile reused across the tile, so it runs at high throughput; but only the outputs active in $M_{out}$ (red) are required, and the throughput spent on the rest (dotted) is wasted.**

computes

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{x}[\mathcal{N}_i(p)]\,\mathbf{f}_i, \tag{2}$$

where $\mathcal{N}_i(p)$ is the $i$-th position in $\mathcal{N}(p)$ and $\mathbf{f}_i \in \mathbb{R}^{C_{in}\times C_{out}}$ the corresponding weight slice. How unrequested positions are treated is policy-specific: event and temporal execution retains prior state, while learned gating merges results into a residual. Coverage is therefore the operator-level correctness condition; the policy determines the treatment of all other positions. Beyond coverage, a path may issue inactive or $\perp$ slots; dense convolution is the extreme where $\{c \in C : c \neq \perp\} = \mathcal{G}$. The policy fixes the accuracy and required work.

## 2.2 The Sparsity-to-Latency Gap




(a) Latency per batch (b) Relative throughput

**Figure 3: YOLOv8m [23] at batch 4 on WildTrack 640 and RTX 3080 (FP32) under the DeltaCNN [34] temporal difference masks, comparing cuDNN (Dense), DeltaCNN tile skipping, and MMCV gather-scatter [6]; all paths consume the same masks. (a) End-to-end latency per batch step; the dashed line (Ideal) is dense latency scaled by the active ratio of the convolution work. (b) Throughput normalized to dense raw throughput. Bar height is raw throughput $T$; the colored share is effective throughput $T_{\text{eff}}$ (labeled) and the dotted share is spent on inactive positions.**

Existing systems execute a mask along one of two paths. *Tile skipping* [10, 20, 27, 33, 36, 46] (Figure 4) inherits the dense pipeline's tile-based computation without additional metadata passes, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [34] fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, to cut wasted work. *Gather-scatter* [25, 48] (Figure 5) instead abandons the dense pipeline: it extracts exactly the active coordinates into a packed representation, runs a separate GEMM, and scatters the results back, removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [44] adopts a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

A policy fixes the active ratio $\alpha$, defined as the fraction of output-grid positions marked active, whereas a concrete execution path determines the useful-compute ratio $\eta$, defined as the fraction of

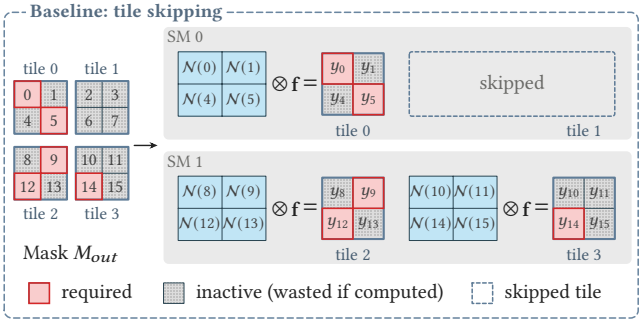



**Figure 4: Tile skipping on the example of Figure 2. It keeps the dense tiled pipeline, so each issued tile runs at dense throughput without extra passes, and skips only tiles with no active position. A partially active tile still runs in full, so its inactive (dotted) outputs are wasted computation.**

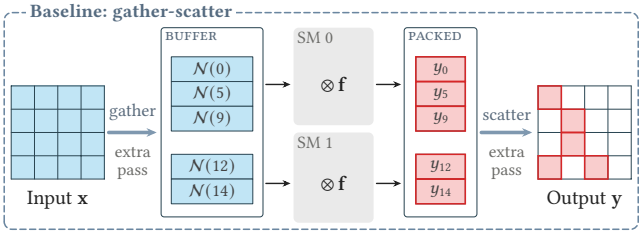



**Figure 5: Gather-scatter on the example of Figure 2. To compute only active outputs, it copies their receptive fields into a packed buffer (gather) so that a GEMM can process them as contiguous rows, then writes the packed results back into y (scatter). No computation is wasted, but the two extra passes over memory, not the computation, dominate its latency.**

issued slots that correspond to required outputs:

$$\alpha = \frac{\|M_{out}\|_0}{|\mathcal{G}|}, \quad \eta = \frac{\|M_{out}\|_0}{|C|}. \tag{3}$$

For $|C| > 0$, an ideal path achieves $\eta = 1$ by issuing only required outputs; dense convolution, which issues the entire grid, has $\eta \approx \alpha$. An execution path also determines its throughput $T$, which combines with $\eta$ to yield the effective throughput $T_{\text{eff}}$:

$$T = \frac{2|C|k^2 C_{in} C_{out}}{t}, \quad T_{\text{eff}} = \eta T. \tag{4}$$

where $t$ denotes the latency of the execution path, including auxiliary mask and coordinate processing, $k$ is the filter kernel size, and $C_{in}$ and $C_{out}$ are the input and output channel counts respectively. The factor of two counts each multiply-accumulate as two FLOPs. Consequently, $T_{\text{eff}}$ directly characterizes the processing rate of required outputs and therefore determines the realized latency.

Figure 3 makes this concrete on RTX 3080 for YOLOv8m [23] at batch 4 under temporal difference masks [34], where 82.2% of convolution work is unnecessary. Tile skipping holds throughput at 41.9% of dense but at a useful-compute ratio of only 67.4%; gather-scatter lifts the useful-compute ratio to one, yet extraction traffic

collapses its throughput to 31.7% of dense. Neither path efficiently converts the GPU's capability into effective throughput.

## 2.3 Distinction from Related Sparse Execution

WISEConv differs from other sparse-execution work in representation and sparsity type. 3D sparse-convolution systems [8, 19, 39, 42, 43, 49] operate on sparse point-cloud coordinates; sparse matrix multiplication [14, 17, 35] targets structural sparsity in graphs or pruned weights; structured weight pruning [29] exploits fixed patterns; sparse-format compilers [24, 50] encode sparsity in tensor formats. DPACS [18] targets the same dynamic 2D spatial sparsity but designs a custom dataflow accelerator to handle the resulting irregular access patterns.

Coordinate-indexed implicit GEMM is not new. Systems for 3D sparse convolution [39, 43] redirect the position dimension of a tiled GEMM through an index array derived from coordinate maps, and CUTLASS fuses such gathers into GEMM [1]. Because a point cloud is itself a sparse coordinate set, building the index array takes hashing, sorting, and global scans that often outlast the GEMM [43, 49], and every reduction step gathers its input rows through the array. Neither cost arises in masked 2D convolution. The output grid is regular and the mask is a bitmap, so a worklist takes one compaction pass over the mask (§4.2); the input stays a dense grid, so the indirection fixes only each row's base address and leaves the reduction identical to dense, and the residual gap is a per-layer fixed cost rather than lost throughput (§3). Other kernels that load rows by index, such as mixture-of-experts grouped GEMM [16, 41] and the Blackwell TMA row gather [30], use the indirection to avoid a copy but do not characterize its cost against the dense kernel.

Selecting kernels by problem size is equally established. Compilers and inference engines for dynamic shapes tune kernels over shape ranges and dispatch among them at run time [31, 38, 52], GEMM libraries choose a schedule per problem size [37], and DA-MoE selects fused mixture-of-experts kernels on the device from each layer's token distribution [22]. These systems serve problems whose size changes per request or batch and whose kernels run long enough to absorb a selection step. Masked convolution in high-rate perception changes its size at every layer of every frame, and its layers last microseconds, so each decision point competes with the gain it buys (§3.5).

# 3 Characterizing Worklist-Driven Convolution

Before presenting WISEConv, we characterize what worklist-driven tiled convolution achieves and where it falls short. We study the compute stage (a dense tiled implicit-GEMM kernel redirected to compute only positions from a worklist; §4.4) and the construction stage (a mask-to-worklist pass) separately, so that each finding belongs to the execution model, not to a particular implementation. FP16 uses `mma.sync` tensor cores; FP32 is a SIMT kernel of the same structure. Each configuration is timed in isolation inside CUDA graphs on RTX 3080 (68 SMs) and Jetson AGX Orin (16 SMs) at locked clocks, using YOLOv8m [23] at batch 1 on WildTrack [5] unless noted otherwise. Dense time is the faster of our compute kernel with every position in the worklist (best configuration) and cuDNN's fastest non-Winograd, non-FFT algorithm.

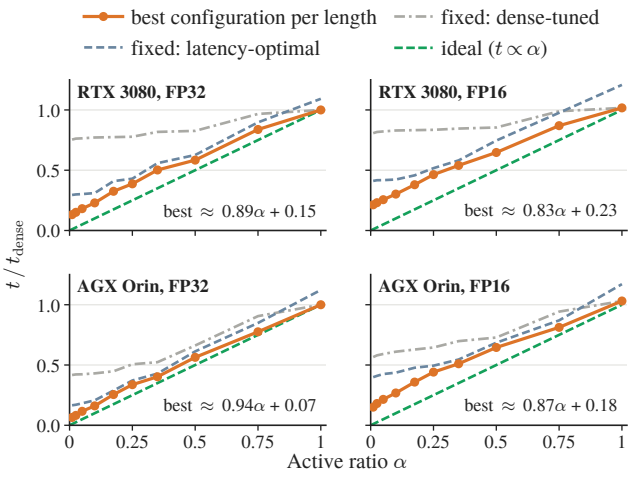



**Figure 6: Convolution time over dense time, averaged over the layers of YOLOv8m, against the active ratio. A configuration is one point of the dense kernel's configuration space (tile shape, warp layout, pipeline depth, and reduction split [1]). The per-length best takes the fastest configuration at each worklist length; the two fixed configurations keep one configuration for every length: the latency-optimal one has the lowest expected time over a uniform $\alpha$, and the dense-tuned one is the fastest at $\alpha = 1$.**

## 3.1 Affine Latency Requires Following the Length

A worklist kernel issues only active outputs ($\eta \approx 1$), so proportional speedup means keeping dense throughput. When we grant the kernel its best configuration at every worklist length, its latency averaged across layers is affine in $\alpha$, well fitted by $t/t_{\text{dense}} = a\alpha + b$. On RTX 3080, $a = 0.89$, $b = 0.15$ (FP32) and $a = 0.83$, $b = 0.23$ (FP16); on AGX Orin, $a = 0.94$, $b = 0.07$ (FP32) and $a = 0.87$, $b = 0.18$ (FP16), with per-layer $R^2$ medians of 0.985–0.995 (Figure 6). The slope $a$ matches the dense marginal cost; the entire departure from proportional is the intercept $b$, a per-layer fixed cost analyzed in §3.4.

This affine behavior holds only when the configuration follows the worklist length. At $\alpha \leq 0.05$, the dense-tuned configuration (the fastest at $\alpha = 1$) runs 6.8× (FP32) and 4.4× (FP16) slower than the per-length best on RTX 3080, and 6.2× and 3.6× on AGX Orin (Figure 6): large tiles amortize weight loads over many rows, but when rows are few, configurations that expose more parallel work (smaller tiles or a split reduction) shorten the chain each SM executes. The latency-optimal configuration, the single configuration with the lowest expected time over a uniform $\alpha$, narrows the gap to 2.0–2.6× at $\alpha \leq 0.05$, at the price of running 9.2–18.7% slower than dense at $\alpha = 1$, and remains 11.2–16.5% slower than the per-length best in expected time.

**Insight 1.** *The latency of worklist-driven tiled convolution is affine in the active ratio at dense marginal cost only when the tile configuration follows the worklist length.*

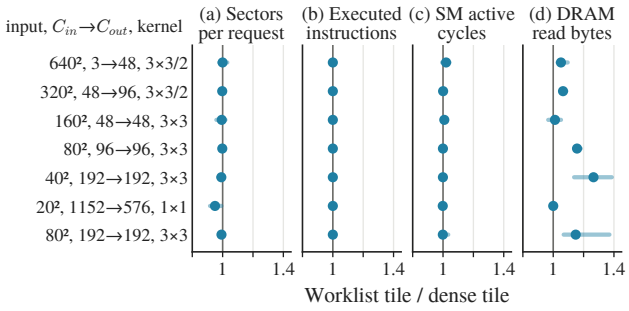



**Figure 7: A worklist tile against a dense tile of the same configuration on representative layers (Nsight Compute, cold L2; bar: range over the two GPUs and two precisions, dot: median). (a) Global-load sectors per request of the median frame. (b)–(d) Cost of one more tile: the slope over the tile count of six worklist prefixes over that of six dense prefixes, so per-launch costs such as the cold weight reads cancel.**

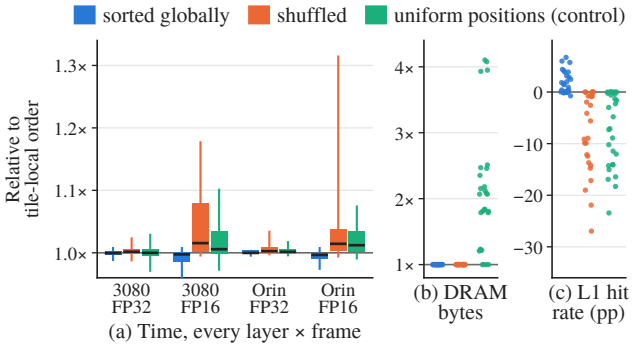



**Figure 8: The worklist in other orders, relative to the tile-local order that construction emits (same length and tile); uniform positions are a control without the spatial clustering of the mask. (a) Every layer and captured frame (box: 25th–75th percentile, whiskers: 5th–95th, line: median). (b), (c) Representative layers, one dot per layer, GPU, and precision (cold L2).**

## 3.2 Why Per-Tile Throughput Matches Dense

The worklist changes only the base address of each row; under a channels-last layout, the reduction loop over $k^2$ taps and channel slices keeps the dense kernel's addressing, so channel coalescing and weight-tile reuse are untouched. Counters confirm it (Figure 7). We compare the cost of one more tile, the slope over the tile count, because a single launch also carries fixed costs (cold weight reads, first misses) that a dense launch spreads over hundreds of tiles and a worklist launch over a few; those costs form the intercept of §3.4. Per added tile, a worklist tile executes the same instructions as a dense tile on every layer, GPU, and precision, and spends 0.98–1.04× its SM cycles; sectors per request stay within 0.92–1.03× of dense. It reads more DRAM bytes, up to 1.39× (median 1.07×), because scattered positions share fewer receptive fields with their neighbors; the $1 \times 1$ layer, whose receptive fields do not overlap, reads exactly as much as dense. The extra reads do not lengthen the tile: its SM cycles stay at dense, so they are hidden behind its computation.

**Insight 2.** *Irregularity is confined to row-level indirection outside the reduction loop, leaving channel-contiguous access and weight-tile reuse identical to dense execution.*

## 3.3 Order Is a Second-Order Effect

Scattered worklist entries could hurt L1 reuse by placing positions with non-overlapping receptive fields into the same tile. Construction emits the worklist in a tile-local order (§4.2): dense traversal order within each construction tile, arrival order across tiles. We compare it with the same worklist sorted globally in dense traversal order and randomly shuffled. Across about 7,900 layer–frame samples per GPU and precision, global sorting runs at 0.997–1.000× the tile-local order (median) and saves at most 1.1% of model time; shuffling runs at 1.002–1.015× (Figure 8a). All three orders read the same DRAM bytes (within 0.1%); shuffling only lowers the L1 hit rate, by up to 27 points (Figure 8b, c). This costs little on RTX 3080 FP32 but up to 1.31× (95th percentile) on AGX Orin FP16, where the low-intensity early layers expose the extra L1-to-L2 traffic to Orin's lower L2 bandwidth. Spatial clustering, not order, sets DRAM traffic: the same number of positions drawn uniformly reads up to 4.1× the bytes.

**Insight 3.** *Worklist order leaves DRAM traffic unchanged; a tile-local order is within 1.1% of a global order.*

## 3.4 The Residual Gap Is a Per-Layer Fixed Cost

The intercept $b$ is the time of a launch that computes almost nothing: a launch that computes a single tile already takes 69–80% of $b$ (Figure 10a), so $b$ is mostly the time to launch the kernel and finish one tile. Without work, a launch spends 65–81% of its stall cycles on constant-cache misses (the kernel parameters are cold); one tile adds barriers and on-chip latency, while DRAM/L2 latency stays at 6–23% (median; Figure 10a). The intercept is 10.0–17.4% of a median layer's dense time, except 5.6% on AGX Orin FP32, where each layer is longer relative to the launch.

Construction adds a second fixed cost. Each construction pass takes 1.6–4.4 $\mu$s on RTX 3080 and 3.8–27.2 $\mu$s on AGX Orin, almost independent of activity, while the convolution it feeds shrinks with $\alpha$. Its share of model time therefore rises as activity falls (Figure 10b). On RTX 3080 FP32 the share grows from 3.8% at $\alpha = 0.5$ to 11.4% at $\alpha = 0.1$; on the faster FP16 path it reaches 39.3% at $\alpha = 0.01$.

**Insight 4.** *The residual gap comes from per-layer fixed costs independent of activity: launch latency in the compute intercept, and near-constant construction time.*

## 3.5 Following the Length Is Not Free

Insight 1 requires the configuration to follow the worklist length, but the length exists only on the device and changes at every layer of every frame. Every mechanism that selects a kernel from it pays per decision point. Measured in isolation, reading the length back to the host costs 7.5 $\mu$s per layer on RTX 3080 and 20.8 $\mu$s on AGX

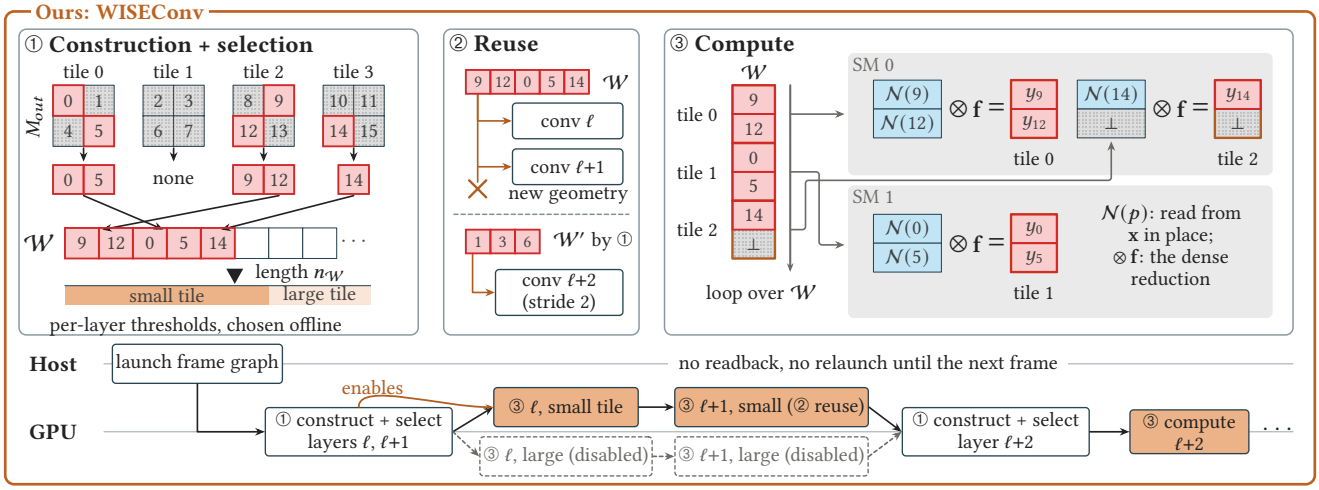



**Figure 9: WISEConv on the running example of Figure 2. ① One pass over $M_{out}$ compacts each construction tile into a segment, appends the segments to $\mathcal{W}$ in arrival order, and selects from $n_{\mathcal{W}}$ the tile configuration of every layer that reads $\mathcal{W}$. ② Operators with the same geometry reuse $\mathcal{W}$; a stride-2 convolution gets a new one. ③ A persistent grid loops over $\mathcal{W}$ in tiles of the selected size, reading receptive fields from x in place. Bottom: one host launch per frame; on the device, construction enables the selected candidates of layer $\ell$ and of the reusing layer $\ell$+1 together.**

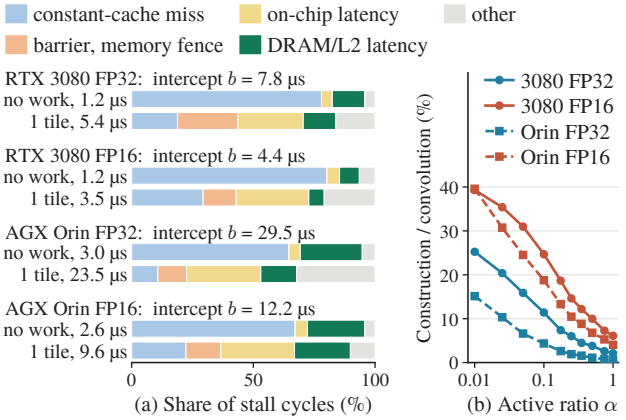



**Figure 10: The two per-layer fixed costs. (a) Stall cycles by cause and time of a compute launch with no work or one tile, against the intercept $b$ (medians over representative layers; on-chip latency: short scoreboard and fixed-latency waits). (b) Construction over convolution time.**

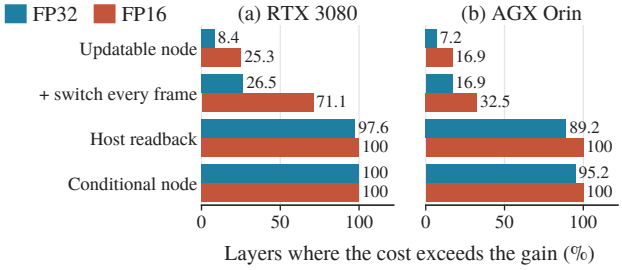



**Figure 11: Share of the layers of YOLOv8m on which a selection mechanism costs more than it saves. A layer's saving is how much faster per frame the per-length best configuration runs than the best single configuration, on the layer's traced worklist lengths. The updatable node is shown without and with a candidate switch in every frame.**

Orin, and an IF conditional graph node 9.5 and 32.4 $\mu$s; a device-updatable kernel node costs 0.8 $\mu$s per layer plus 0.8 $\mu$s per candidate switch (1.7 plus 2.0 $\mu$s on AGX Orin). The median per-layer gain of following the length is 1.3–2.7 $\mu$s on RTX 3080 and 5.0–12.3 $\mu$s on AGX Orin. Readback and conditional nodes exceed the gain on 89–100% of layers (Figure 11), the updatable node on 7–25% without switching and on up to 71% when it switches every frame. A candidate pays off only where the gain exceeds this cost, so the mechanism's cost, not the oracle's gain, limits adaptation.

**Insight 5.** *Every mechanism for following the worklist length incurs a per-decision-point cost often comparable to the per-layer gain from adaptation.*

Insights 4 and 5 together mean that current hardware cannot reach the oracle affine line without trading selection cost against adaptation gain; §6.5 discusses the hardware support that would remove this trade-off.

# 4 WISEConv Design

## 4.1 Overview

WISEConv exploits the fact that dense tiled convolution is driven by output coordinates: each coordinate determines the receptive-field reads and output write, while the reduction over channels

**Algorithm 1** Tile-Local Worklist Construction

**Require:** Input mask $M_{in}$, output grid $\mathcal{G}$
**Ensure:** Mask $M_{out}$, worklist $\mathcal{W}$, length $n_{\mathcal{W}} = |\mathcal{W}|$

1: $n_{\mathcal{W}} \leftarrow 0$
2: **for** each construction tile $\tau \subset \mathcal{G}$ **in parallel do**
3: **for** each position $p \in \tau$ **in parallel do**
4: active$(p) \leftarrow \exists r \in \mathcal{N}(p) : M_{in}[r] = 1$
5: $M_{out}[p] \leftarrow$ active$(p)$
6: **end for**
7: $\mathcal{P}_\tau \leftarrow$ Compact$(\tau, M_{out})$
8: $n_\tau \leftarrow |\mathcal{P}_\tau|$
9: base $\leftarrow$ AtomicAdd$(n_{\mathcal{W}}, n_\tau)$
10: **for** each $0 \le j < n_\tau$ **in parallel do**
11: $\mathcal{W}[\text{base} + j] \leftarrow \mathcal{P}_\tau[j]$
12: **end for**
13: **end for**

and kernel offsets remains regular. Replacing the dense coordinate sequence with an active-output worklist introduces position-level selectivity into the tiled pipeline without altering the reduction datapath (Figure 9).

The characterization shows that a worklist tile keeps the per-tile throughput of dense execution (§3.2). What remains between this selectivity and end-to-end latency reduction are the costs isolated by Insights 4 and 5: each construction pass costs nearly constant time whatever the activity (§3.4), and following the worklist length requires a choice of configuration that only the device can make (§3.5). Figure 9 shows how WISEConv handles both on the running example. Construction (①, §4.2) builds the worklist from $M_{out}$ in one pass over mask metadata, and the same pass selects the tile configuration of every layer that reads the worklist (§4.5). Later operators with the same geometry reuse the worklist (②, §4.3), so neither construction nor selection repeats at every layer. The compute stage (③, §4.4) consumes the worklist in a persistent grid that reads its length from device memory. All stages run in one per-frame CUDA graph (bottom of Figure 9): construction enables, on the device, the selected candidate of every layer that reads the worklist (the small-tile nodes of $\ell$ and $\ell$+1), so the frame needs no host synchronization between layers.

## 4.2 Tile-Local Worklist Construction

Construction turns $M_{in}$ into $M_{out}$ and the worklist $\mathcal{W}$ in one pass over the layer's output grid, reading and writing only mask metadata, never features or weights (Algorithm 1). The pass partitions the grid into construction tiles, one per CTA. Each CTA derives its tile's entries of $M_{out}$ by Eq. 1 (lines 3–6; a learned gate supplies them directly), compacts the tile's active positions in dense traversal order (line 7), and reserves a contiguous segment of $\mathcal{W}$ with one atomic add on a device-resident length counter $n_{\mathcal{W}}$ (line 9). The buffer holds $|\mathcal{G}|$ entries, the static upper bound on $n_{\mathcal{W}}$, so construction allocates nothing at run time and $n_{\mathcal{W}}$ never leaves the device. Every active position lands in exactly one segment, so the worklist is exact: $n_{\mathcal{W}} = \|M_{out}\|_0$.

The resulting order is tile-local: dense within each segment, arrival order across segments. A globally ordered worklist would need a device-wide prefix scan or a second pass, and order has a negligible effect on the compute stage (Insight 3). What remains is the cost of the pass itself: it scans the whole grid, so it is fixed by the layer shape while the convolution it enables shrinks with activity and channel width.

## 4.3 Cross-Operator Worklist Reuse

WISEConv therefore builds a worklist only where the active set can grow. Layer $l+1$ reuses the worklist of layer $l$ when it keeps the output grid and cannot activate a position outside layer $l$'s active set:

$$\mathcal{G}^{l+1} = \mathcal{G}^{l}, \qquad \{p \mid M_{out}^{l+1}[p] = 1\} \subseteq \{p \mid M_{out}^{l}[p] = 1\}. \quad (5)$$

The condition depends on operator types, not on mask values ($1\times1$ convolutions, activations, and normalization satisfy it), so model conversion marks reuse edges once. A reused worklist need not be exact. To track where issued work goes, we factor the useful-compute ratio of Eq. 3 into a construction and a compute term:

$$\eta = \underbrace{\frac{\|M_{out}\|_0}{n_{\mathcal{W}}}}_{\eta_{\text{construct}}} \cdot \underbrace{\frac{n_{\mathcal{W}}}{|C|}}_{\eta_{\text{compute}}}. \quad (6)$$

A freshly constructed worklist has $\eta_{\text{construct}} = 1$. When the upstream policy narrows a mask, as DeltaCNN's update truncation does [34], the reused worklist becomes a superset ($\eta_{\text{construct}} < 1$); the extra positions cost compute time but do not affect the output, because only positions active in the layer's own $M_{out}$ are written. A new grid or a receptive field wider than one position marks a rebuild point. Rebuild points recur at every $3\times3$ convolution, so the worklist length changes from layer to layer as well as from frame to frame, and the compute stage must consume a length that only the device knows.

## 4.4 Worklist-Driven Tiled Compute

The compute stage keeps the dense kernel's implicit-GEMM mapping and changes only which output positions a tile covers. Dense convolution maps to a GEMM with $M$ output positions, $N = C_{out}$, and reduction depth $k^2 C_{in}$; WISEConv sets $M = n_{\mathcal{W}}$, so GEMM row $i$ is output position $\mathcal{W}[i]$. A configuration $q$ is one point of the dense kernel's configuration space: the tile shape $B_M \times B_N$, the reduction step $B_K$, and the remaining tiling parameters such as warp layout, pipeline depth, and a reduction split within or across CTAs [1]. WISEConv adds no parameter to this space.

Because $n_{\mathcal{W}}$ exists only on the device, the host cannot launch the right number of tiles. Algorithm 2 instead launches a persistent grid of $P$ CTAs, sized by the configuration and the GPU alone, which reads $n_{\mathcal{W}}$ from device memory and strides over the output tiles it implies (line 2); CTAs beyond the tile count exit immediately, so one static launch serves every length and every frame. Each tile reads its $B_M$ worklist entries once (line 4); the reduction (lines 6–12) then walks the kernel offsets and channel slices exactly as dense execution does. Because the input stays a dense grid, every neighbor of an active position lies at a fixed offset from that position, so the worklist sets only each row's base address and the addresses inside the reduction loop remain affine. The irregularity is $B_M$ indexed loads per tile outside the reduction, while channel-contiguous loads

**Algorithm 2** Worklist-Driven Tiled Compute (configuration $q$)

**Require:** Input $\mathbf{x}$ on the dense grid, weights $\mathbf{f}$, output mask $M_{out}$, worklist $\mathcal{W}$ and length $n_{\mathcal{W}}$ in device memory; tile count $N_t = \lceil n_{\mathcal{W}}/B_M \rceil \lceil C_{out}/B_N \rceil$; $P$ CTAs
**Ensure:** Output $\mathbf{y}$ at every active position

1: **for** each CTA $g = 0, \ldots, P-1$ **in parallel do**
2: **for** $t = g,\ g+P,\ g+2P, \ldots$ while $t < N_t$ **do**
3: $(b_m, b_n) \leftarrow$ output tile $t$
4: $R \leftarrow \mathcal{W}[b_m B_M : (b_m+1)B_M]$ ▷ *active positions*
5: $\mathbf{Y} \leftarrow \mathbf{0}_{B_M \times B_N}$
6: **for** each receptive-field offset $i = 1, \ldots, k^2$ **do**
7: **for** each channel slice $s = 1, \ldots, \lceil C_{in}/B_K \rceil$ **do**
8: $\mathbf{X} \leftarrow$ load $\mathbf{x}[\mathcal{N}_i(p)]$ for $p \in R$, slice $s$
9: $\mathbf{B} \leftarrow$ load $\mathbf{f}_i$, slice $s$, column block $b_n$
10: $\mathbf{Y} \leftarrow \mathbf{Y} + \mathbf{XB}$
11: **end for**
12: **end for**
13: write $\mathbf{Y}$ to $\mathbf{y}$ at each $p \in R$ with $M_{out}[p] = 1$
14: **end for**
15: **end for**

and weight-tile reuse are those of the dense kernel (Insight 2). The epilogue stores only positions active in $M_{out}$ (line 13), so the padding of the last tile and the superset entries inherited through reuse produce no writes. That padding is the only issued work beyond the worklist: $\eta_{\text{compute}} = n_{\mathcal{W}}/(\lceil n_{\mathcal{W}}/B_M \rceil B_M)$.

What a static grid cannot fix is the configuration itself: the fastest $q$ depends on $n_{\mathcal{W}}$ (Insight 1).

### 4.5 Cost-Aware Tile Selection

Following $n_{\mathcal{W}}$, however, costs time at every decision point (Insight 5). WISEConv therefore decides per layer whether, and how finely, to follow the length, by weighing the expected gain of each additional candidate configuration against its cost.

For layer $l$, an offline sweep on the target GPU measures the time $t_q(n)$ of every configuration $q$ in the space $\mathcal{Q}$ across worklist lengths $n$; $t^*(n) = \min_{q \in \mathcal{Q}} t_q(n)$ is the per-length best. A selection $S$ with $K$ candidates $q_1, \ldots, q_K$ and thresholds $0 = \theta_0 < \theta_1 < \cdots < \theta_K = |\mathcal{G}| + 1$ runs $q_S(n) = q_k$ for $\theta_{k-1} \le n < \theta_k$ and pays the GPU's per-layer cost of choosing among $K$ candidates, $c(K)$ with $c(1) = 0$, measured once on the same GPU. Without a prior on the deployment's activity, the selection assumes $n$ uniform over $[1, |\mathcal{G}|]$ and maximizes the geometric-mean speedup, that is, minimizes the expected log-slowdown relative to the per-length best:

$$\hat{S} = \arg\min_{K,S} \mathbb{E}_n \left[ \log \frac{t_{q_S(n)}(n) + c(K)}{t^*(n)} \right]. \tag{7}$$

Because $n$ is a scalar, the candidates partition a single range. On the sweep's lengths $n_1 < \cdots < n_J$, with $L_q(u)$ the log term of Eq. 7 at $n_u$ and $w_u$ the probability mass of $n_u$'s interval, the best expected cost $F_k(j)$ of covering the first $j$ lengths with $k$ candidates obeys

$$F_k(j) = \min_{i<j} \left[ F_{k-1}(i) + \min_{q \in \mathcal{Q}} \sum_{u=i+1}^{j} w_u L_q(u) \right], \qquad F_0(0) = 0, \tag{8}$$

and evaluating $F_K(J)$ for each $K$ yields $\hat{S}$ exactly. The gain of each added candidate shrinks while $c(K)$ grows, so $K$ is an output of the trade-off rather than a design constant.

At run time the decision must happen where $n_{\mathcal{W}}$ first exists, without host involvement. The kernel that writes a layer's worklist learns $n_{\mathcal{W}}$ when its last CTA finishes, compares it with the thresholds of every layer that reads this worklist (the layer and the operators that reuse it), and switches each to its selected candidate, when it differs from the current one, before the compute launches. The design does not depend on how the switch is performed: $c(K)$ is the cost of the cheapest device-side mechanism the GPU offers (§5). As a result, $K$ differs across layers, GPUs, and precisions: layers whose optimum shifts little keep one configuration and pay nothing for selection; §6.4 shows that both ignoring the cost and fixing one configuration fall short of this selection.

## 5 Implementation

We implement WISEConv as a PyTorch extension with a Python frontend and a CUDA backend written in C++. A model-conversion pass replaces supported modules, preserves their parameters and weights, and statically marks worklist-reuse edges and rebuild points. The compute backend is a family of worklist-driven implicit-GEMM kernels in the style of CUTLASS [1], covering FP32 (SIMT) and FP16 (tensor core) arithmetic with NHWC tensors, all convolution variants of the evaluated models, and mask-aware auxiliary operators. Each frame runs as one CUDA graph. Selection uses the lowest-latency device-side mechanism CUDA currently offers, device-updatable kernel nodes (CUDA 12.4): each candidate is a node of the graph, and the worklist producer enables the selected node and disables the previous one only when the selection changes. Every kernel agrees numerically with cuDNN at each active output position.

## 6 Evaluation

### 6.1 Experimental Setup

We ask three questions: whether WISEConv turns spatial sparsity into end-to-end latency and energy reduction across GPU classes (§6.2), where its remaining time goes (§6.3), and whether each co-designed component is necessary (§6.4).

*Platforms.* We evaluate six NVIDIA GPUs that span datacenter, desktop, laptop, and embedded classes: A100, RTX 4090, RTX 3080, RTX 4070 Laptop, Jetson AGX Orin, and Jetson Xavier NX. GPU clocks are locked on every platform so that latencies are stable across runs. On the two Jetson boards, where the energy budget constrains deployment, we also report per-frame energy from the board's input power.

*Workloads.* The three workloads (Table 1) cover the three mask sources of §2.1: event masks, temporal-difference masks, and learned spatial gates. Each mask policy follows its upstream method; where it needs thresholds, they are tuned on data disjoint from the evaluation set.

*Baselines.* *Dense* runs the original model with PyTorch and cuDNN [7] autotuning: it computes every position and carries none of the masking machinery (no mask generation, propagation, or temporal state).

**Table 1: Workloads.**

| | **FireFlowNet** [32] | **YOLOv8m** [23] | **DynConv** [44] |
|---|---|---|---|
| Task | Optical flow | Detection | Human pose |
| Mask source | Ev-Conv [10] | DeltaCNN [34] | Learned gates [44] |
| Dataset | MVSEC [53] | WildTrack [5] | MPII [2] |
| Input | 256×256 | 640×640 | 256×256 |
| Parameters | 0.056M | 25.9M | 6.89M |
| Eval. frames | 196.8K | 251.7K | 2,958 |

*Tile skipping* uses DeltaCNN's kernels [34], and *gather-scatter* uses MMCV's masked convolution [6] (DynConv's own CUDA backend [44] for pose). All paths run the same weights and inputs, and the three masked paths the same layer masks, so they differ only in the masked-convolution backend. Every path except gather-scatter replays as a CUDA graph; gather-scatter reads each layer's active count on the host to size its buffers and its GEMM, so it must run eagerly. The baselines implement only FP32, so FP16 compares WISEConv with Dense alone. We report WISEConv's speedup over the fastest baseline on each workload and platform.

*Protocol.* Latency is the GPU time of one frame measured with CUDA events; it includes worklist construction and all non-convolution operators. YOLOv8m processes whole videos continuously, so its temporal masks evolve as in deployment, at batch 1 (YOLOv8m b1: a single latency-bound camera, our primary setting) and batch 4 (YOLOv8m b4: four concurrent camera streams). WISEConv's per-device tuning (§4.5) runs once offline and is excluded. Task accuracy is reported to confirm that the sparse paths agree.

## 6.2 End-to-End Performance

*Latency.* WISEConv is the fastest path in all 44 groups of workload, platform, and precision (Figure 12). In FP32 it runs 1.49–3.23× faster than the fastest baseline of each group (geometric mean 2.06×), and in FP16, where Dense is the only baseline, 1.27–4.25× faster (2.10×), 2.08× over all 44 groups. No baseline is consistently second: tile skipping is the fastest baseline in 13 of the 24 FP32 groups, gather-scatter in 6, and Dense in 5. Gather-scatter varies most, because its per-layer extraction, scatter, and host synchronization do not shrink with the active ratio. In 12 FP32 groups, mostly FireFlowNet and DynConv, it runs 1.85–12.1× slower than Dense and extends beyond the axis (red crosses). On YOLOv8m b4, four streams amortize these costs and make it the fastest baseline on four of the six GPUs, where WISEConv still runs 1.69–2.39× faster.

The distance to the ideal line follows worklist length, as Insight 4 predicts. As a geometric mean over GPUs and precisions, WISEConv runs at 1.61× the ideal latency on YOLOv8m b4, whose median worklist holds 1,382 entries, and at 2.78× on DynConv, whose median worklist holds 43 (Table 4), so per-layer fixed costs dominate its frame.

*Energy.* Shorter frames translate into lower energy (Table 2). WISEConv is the lowest-energy path for every workload on both Jetson boards: in FP32 it uses 32.8–51.8% less energy per frame than the lowest-energy baseline, and across both precisions 60.8–82.5% less than Dense.

**Table 2: Board energy per frame (mJ) on the Jetson platforms. Parentheses give each masked path's change relative to Dense of the same precision; WISEConv is in bold.**

| | | FireFlowNet | YOLOv8m b1 | YOLOv8m b4 | DynConv |
|---|---|---|---|---|---|
| AGX Orin | FP32 Dense | 165 | 1508 | 1463 | 951 |
| | FP32 Tile skipping | 79.9 (−51.5%) | 752 (−50.1%) | 572 (−60.9%) | 467 (−50.9%) |
| | FP32 Gather-scatter | 205 (+24.2%) | 791 (−47.6%) | 543 (−62.9%) | 2324 (+144.3%) |
| | FP32 WISEConv | **48.6** (−70.5%) | **363** (−76.0%) | **280** (−80.9%) | **258** (−72.8%) |
| | FP16 Dense | 47.7 | 377 | 352 | 314 |
| | FP16 WISEConv | **17.5** (−63.3%) | **101** (−73.1%) | **61.6** (−82.5%) | **123** (−60.8%) |
| Xavier NX | FP32 Dense | 248 | 2166 | 2135 | 1067 |
| | FP32 Tile skipping | 114 (−54.0%) | 876 (−59.5%) | 837 (−60.8%) | 499 (−53.3%) |
| | FP32 Gather-scatter | 299 (+20.2%) | 1028 (−52.5%) | 818 (−61.7%) | 2519 (+136.1%) |
| | FP32 WISEConv | **76.8** (−69.1%) | **539** (−75.1%) | **450** (−78.9%) | **332** (−68.9%) |

**Table 3: Task accuracy on RTX 3080 (mAP over IoU 0.5–0.95). Dense runs the unmasked model; the three masked paths consume the same layer masks.**

| | | FireFlowNet | YOLOv8m b1 | | DynConv |
|---|---|---|---|---|---|
| | | AEE ↓ | AP50 ↑ | mAP ↑ | PCKh ↑ |
| FP32 | Dense | 1.903 | 0.632 | 0.205 | 88.06 |
| | Tile skipping | 1.888 | 0.639 | 0.209 | 86.68 |
| | Gather-scatter | 1.888 | 0.639 | 0.209 | 86.68 |
| | WISEConv | 1.888 | 0.640 | 0.210 | 86.68 |
| FP16 | Dense | 1.903 | 0.633 | 0.205 | 88.06 |
| | WISEConv | 1.889 | 0.639 | 0.210 | 86.66 |

*Accuracy.* Given the same layer masks, the three masked paths reach the same accuracy (Table 3): AEE and PCKh are identical, and YOLOv8m AP differs by at most 0.1 points, a residual of floating-point reduction order. Where worklist reuse computes a superset of the required positions, the output mask gates writeback, so the extra positions never reach downstream layers. The remaining gap to Dense (1.4 PCKh points lower on DynConv, slightly better AEE and AP elsewhere) is set by the upstream mask policy.

## 6.3 Microbenchmarks

*Effective throughput.* Figure 13 explains the end-to-end result with the two axes of Eq. 4: a masked path is fast only if it is both selective (useful-compute ratio $\eta$) and keeps dense throughput ($T$). The two baselines fail on opposite axes. Tile skipping keeps the dense tile schedule, reaching 17.5–59.9% of Dense raw throughput, but computes every position of an active tile, so $\eta$ is 0.49–0.74. Gather-scatter issues only required positions ($\eta = 1$), but extraction and writeback hold its raw throughput to 2.6–42.0% of Dense. WISEConv keeps $\eta$ at 0.91–1.00 and sustains 52.3–76.0% of Dense raw throughput in FP32 (29.6–76.0% in FP16), the highest effective throughput in every group.

Table 4 locates WISEConv's remaining loss with the decomposition of Eq. 6. Tile-local compaction emits each active coordinate exactly once ($\eta_{\text{construct}} = 100\%$), so all loss is padding in the final tile of each worklist ($\eta_{\text{compute}}$) and it follows worklist length: 99.8% and 98.1% on FireFlowNet and YOLOv8m b4, whose median worklists hold thousands of entries, and 93.6% and 92.1% on YOLOv8m b1 and DynConv, whose shortest worklists (P10) hold 26 and 3 entries.

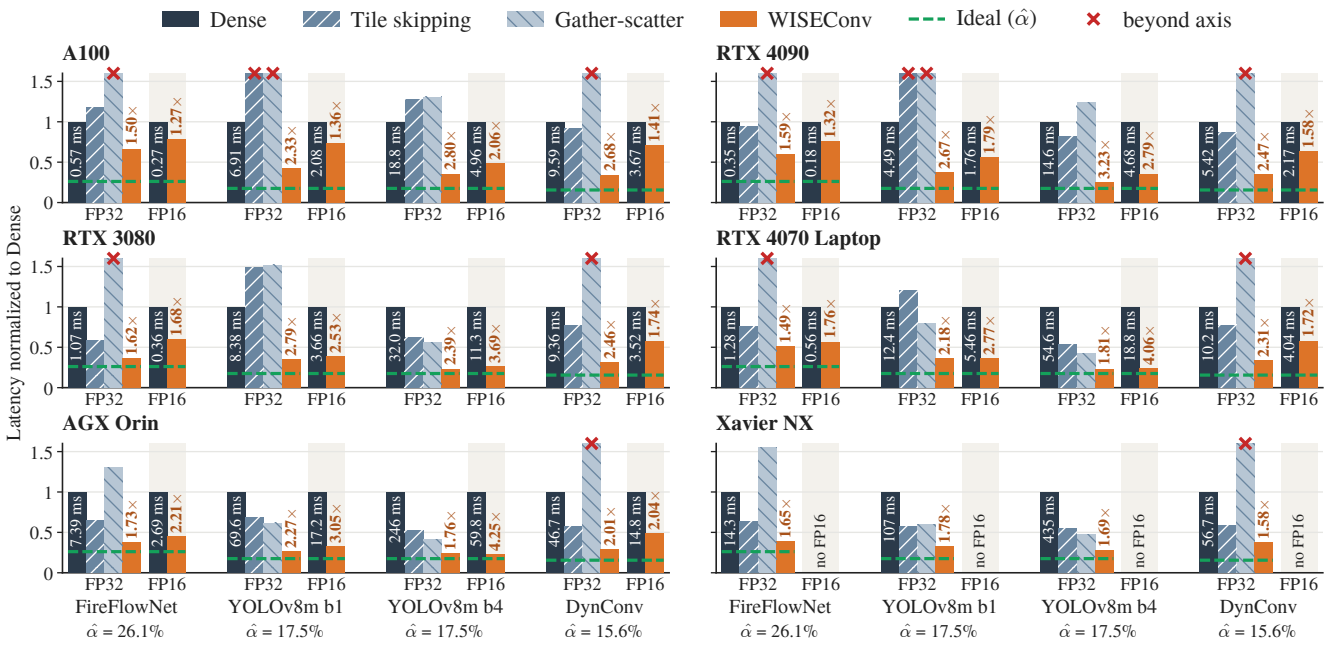



**Figure 12: Full-model latency on six GPUs, normalized to Dense of the same precision (lower is better). Each workload has an FP32 group with all four paths and an FP16 group with Dense and WISEConv, since the baselines have no FP16 path. Dense bars give the absolute latency (per frame; per batch of four for YOLOv8m b4), WISEConv bars give its speedup over the fastest other path of the group, and a red cross marks bars that extend beyond the axis. The dashed line is ideal proportional scaling, the mean active ratio $\hat{\alpha}$ given under each workload.**

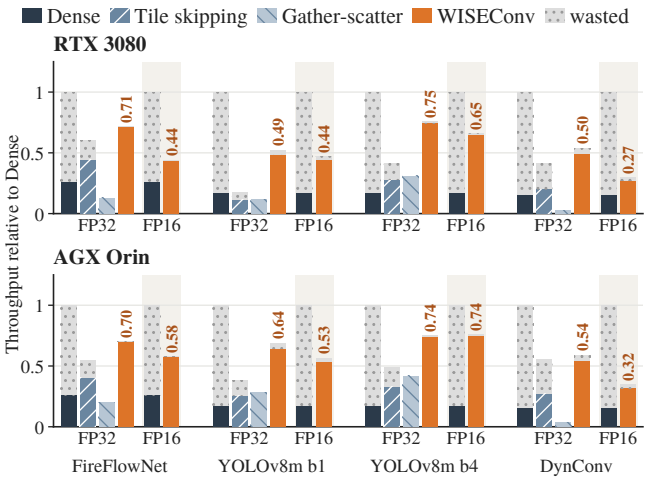



**Figure 13: Effective throughput $\eta T$ (colored) and wasted throughput $(1-\eta)T$ (dotted) of every path, relative to the raw throughput of Dense of the same precision. WISEConv bars give its effective throughput.**

*Latency decomposition.* Figure 14 shows where each path spends a YOLOv8m frame. At b1 in FP32, WISEConv cuts convolution from 7.21 to 2.27 ms at the cost of 0.17 ms of construction, and

**Table 4: WISEConv's $\eta$ decomposition and worklist lengths on RTX 3080 (FP32). The $n_W$ percentiles pool all layers and frames.**

| Workload | $\eta_{\text{construct}}$ | $\eta_{\text{compute}}$ | $\eta$ | $n_W$ | | |
|---|---|---|---|---|---|---|
| | | | | P10 | P50 | P90 |
| FireFlowNet | 100.0% | 99.8% | 99.8% | 1,303 | 15,083 | 33,880 |
| YOLOv8m b1 | 100.0% | 93.6% | 93.6% | 26 | 280 | 1,792 |
| YOLOv8m b4 | 100.0% | 98.1% | 98.1% | 179 | 1,382 | 7,104 |
| DynConv | 100.0% | 92.1% | 92.1% | 3 | 43 | 389 |

keeps the GPU busy because the compute path never returns to the host. Gather-scatter spends the least time in convolution (1.68 ms) because it multiplies compact buffers, but those buffers tie every layer to the host. The gathered buffer and the GEMM's row count both equal the layer's active count, which exists only on the device, so each layer reads the count back before it can allocate and launch; launching for the largest possible count would recompute the dense layer. These round trips leave the GPU idle for 8.26 ms, 64.8% of its frame, and even its busy time alone (4.48 ms) is 1.49× WISEConv's 3.00 ms. At b4, four streams hide most of that idle time and bring it to 4.50 ms per frame, still 2.39× WISEConv's 1.88 ms. In FP16, tensor cores shrink convolution more than the rest of the frame: construction and other operators grow from 24.3% to 39.0% of WISEConv's b1 frame, and its speedup over Dense narrows from 2.79× to 2.53×.

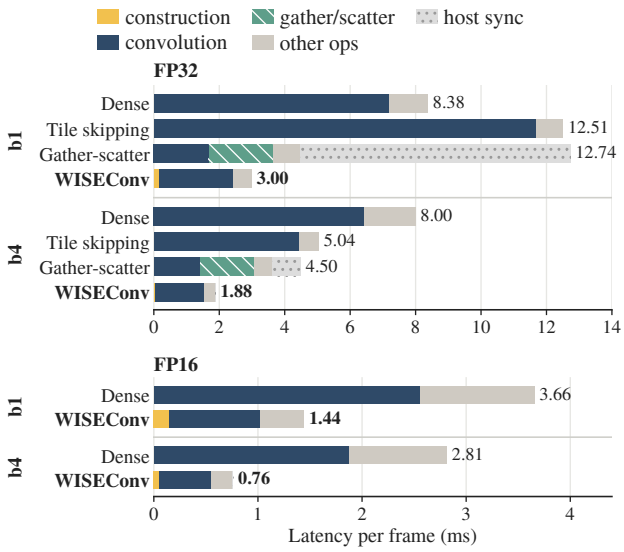



**Figure 14: YOLOv8m latency per frame by stage on RTX 3080, at batch 1 (b1) and 4 (b4), with FP32 and FP16 on separate axes. Stages are each path's kernel shares of GPU busy time, scaled to its end-to-end latency; host sync is the remaining time in the frame, when the GPU idles waiting for the host. Gather-scatter's gather/scatter includes compacting the mask into the index list it gathers by.**

*Speedup across activity.* Grouping the frames of YOLOv8m b1 on RTX 3080 by their active ratio tests the affine behavior end to end. WISEConv's speedup over Dense falls smoothly with activity, from 4.59× for frames with $\alpha < 0.02$ to 1.31× for $\alpha \geq 0.4$ in FP32 (3.14× to 1.43× in FP16), as the affine model with a per-layer intercept predicts (Insights 1 and 4). Neither baseline turns low activity into a speedup: both run slower than Dense at every active ratio. The selection of §4.5 places its gains where the uniform prior expects them: against the speedup-optimal configuration, WISEConv is 1.13× faster on the sparsest frames and at most 4% slower on the densest (FP32).

*Memory overhead.* WISEConv's metadata adds little memory (Table 5). The temporal mask policies of FireFlowNet and YOLOv8m retain previous-frame activations, so every masked path allocates 3.8–6.3× Dense, and WISEConv stays within the range of the other masked paths (128.6 vs. 141.8–197.0 MiB on FireFlowNet). DynConv's learned gates keep no temporal state, and WISEConv allocates 0.1 MiB more than Dense.

**Table 5: Peak live GPU allocation (MiB) on RTX 3080 (FP32).**

| | FireFlowNet | YOLOv8m b1 | YOLOv8m b4 | DynConv |
|---|---|---|---|---|
| Dense | 33.9 | 175.8 | 433.0 | 56.8 |
| Tile skipping | 141.8 | 751.0 | 2701.0 | 53.8 |
| Gather-scatter | 197.0 | 834.8 | 2737.0 | 57.2 |
| **WISEConv** | 128.6 | 763.9 | 2730.4 | 56.9 |

**Table 6: Co-design ablation on YOLOv8m (b1, FP16). Each variant replaces one mechanism of WISEConv. Construction (Cons.), convolution (Conv.), and total are ms per frame; × is latency relative to WISEConv.**

| | RTX 3080 | | | | AGX Orin | | | |
|---|---|---|---|---|---|---|---|---|
| Variant | Cons. | Conv. | Total | × | Cons. | Conv. | Total | × |
| *Construction* | | | | | | | | |
| No reuse | 0.30 | 0.86 | 1.59 | 1.10 | 0.84 | 3.34 | 6.00 | 1.06 |
| Shuffled order | 0.16 | 0.90 | 1.49 | 1.03 | 0.49 | 3.49 | 5.81 | 1.03 |
| *Compute* | | | | | | | | |
| Readback & launch | 0.16 | 0.87 | 9.40 | 6.51 | 0.53 | 3.36 | 28.70 | 5.08 |
| Full-layer grid | 0.16 | 1.03 | 1.61 | 1.12 | 0.48 | 3.79 | 6.08 | 1.07 |
| Dense-tuned config. | 0.16 | 1.89 | 2.47 | 1.71 | 0.48 | 4.91 | 7.23 | 1.28 |
| Speedup-opt. config. | 0.16 | 0.88 | 1.47 | 1.02 | 0.48 | 3.83 | 6.14 | 1.09 |
| Latency-opt. config. | 0.16 | 1.00 | 1.59 | 1.10 | 0.48 | 3.93 | 6.24 | 1.10 |
| Cost-unaware $K_{\max}$ | 0.22 | 0.85 | 1.55 | 1.07 | 0.65 | 3.16 | 5.78 | 1.02 |
| Zero-cost oracle | 0.15 | 0.75 | 1.32 | 0.92 | 0.41 | 2.79 | 4.98 | 0.88 |
| **WISEConv** | 0.16 | 0.86 | 1.44 | 1.00 | 0.48 | 3.34 | 5.65 | 1.00 |

## 6.4 Ablation Study

Table 6 replaces one mechanism at a time on YOLOv8m (b1, FP16) on RTX 3080 and AGX Orin. Rebuilding the worklist at every layer (No reuse) nearly doubles construction time (0.16 to 0.30 ms and 0.48 to 0.84 ms) and adds 9.9% and 6.2% to frame latency. A shuffled worklist lengthens convolution by 4.7% and 4.4% and the frame by 2.8% and 2.6%, as Insight 3 predicts (§3.3). Launching the dense layer's grid, whose CTAs beyond the worklist exit, lengthens convolution by 19.4% and 13.1%, the cost that the persistent grid avoids (§4.4). Reading $n_W$ back to the host at every layer to launch the exact grid runs 6.51× and 5.08× slower, as Insight 5 predicts.

The tile-configuration variants keep WISEConv's selection mechanism (the same layers follow the worklist length, through the same producers and candidate nodes) and change only the candidates, so they isolate what following $n_W$ gains. Fixing every candidate to the configuration fastest at $\alpha = 1$ runs 1.71× and 1.28× slower; fixing it to the configuration with the lowest expected latency over a uniform $n_W$ runs 1.10× slower on both GPUs. The configuration with the best expected speedup (speedup-optimal) comes within 1.6% of WISEConv on RTX 3080 but runs 1.09× slower on AGX Orin, where its convolution takes 3.83 instead of 3.34 ms. The two GPUs differ in how many layers follow the length: the selection (§4.5) adds candidates to a layer only where the gain exceeds that GPU's per-layer selection cost, which holds for 5 of 83 layers on RTX 3080 and for 24 on AGX Orin, whose per-layer gains are larger (Insight 5). Placing four candidates at every layer regardless of this cost (Cost-unaware $K_{\max}$) shortens convolution by 1.7% and 5.4%, but every producer then takes the selecting path, raising construction from 0.16 to 0.22 ms and from 0.48 to 0.65 ms, so the frame runs 1.07× and 1.02× slower; it loses most on RTX 3080, where the gains that the cost model declines are smallest.

The zero-cost oracle runs the same candidates as Cost-unaware $K_{\max}$ on a graph captured for each frame's own $n_W$, with no selection mechanism. It runs at 0.92× and 0.88× WISEConv's latency. Because both pick the same tile for every layer and frame, the gap between them (0.85 vs. 0.75 ms and 3.16 vs. 2.79 ms of convolution, 0.22 vs. 0.15 ms and 0.65 vs. 0.41 ms of construction) is the

cost of the device-side mechanism itself, not of the configuration choice. Current GPUs follow a device-resident length only by reconfiguring the graph (§3.5), so this margin is a hardware limit that software selection cannot remove; a launch predicate evaluated by the hardware would recover it (§6.5).

## 6.5 Discussion

*Limitations.* WISEConv converts spatial sparsity into latency, so its benefit is bounded by how sparse the upstream masks are and by how much of the frame convolution occupies. First, the gain vanishes as the active ratio approaches one: at full activity the per-length best configuration only matches dense execution (§3.1) while construction still runs, so a fully active input runs slightly slower than the dense path. WISEConv consumes masks and makes no claim about their quality or the resulting accuracy; a policy that keeps most positions active should stay on the dense path. Second, WISEConv accelerates convolution only, so convolution's share of frame latency caps the end-to-end speedup. FP16 tensor cores shrink that share, and the remaining non-convolution operators become the next bottleneck, which is why FP16 speedups are narrower than FP32 ones (§6.3). Models dominated by other operators bound the gain further.

*Hardware implications.* The characterization traces the remaining gap to two per-layer costs that current GPUs leave to software. The intercept of the affine model is launch and dependency latency rather than lost bandwidth (Insight 4), and its share grows as inputs become sparser; hardware that shortens the path from launch to the first useful instruction (for example, by staging kernel parameters before the CTA starts) and the dependent chain of a single tile would move sparse execution toward the proportional line. Following the worklist length costs about as much per decision point as it gains (Insight 5), because every existing mechanism either returns to the host or reconfigures the graph. A launch predicate that the hardware evaluates on a device-resident value before allocating CTA resources, or a work distributor that picks the tile shape from the worklist length, would let every layer adapt its configuration at negligible cost.

# 7 Conclusion

We presented WISEConv, a masked-convolution runtime that replaces the dense tiled pipeline's contiguous coordinate stream with an active-output worklist, computing almost exclusively the positions an upstream mask requires while retaining the dense tensor layout and the regular per-position reduction. The construction and consumption stages are co-designed to secure both useful-compute ratio and throughput; our ablation confirms that disabling any single mechanism degrades one axis without improving the other. The resulting higher effective throughput lets WISEConv convert upstream mask sparsity into proportional latency and energy reduction for the latency-critical vision tasks where masked convolution applies.

# Acknowledgments

# References


[1] 2026. CUTLASS: CUDA Templates and Python DSLs for High-Performance Linear Algebra. https://github.com/NVIDIA/cutlass

[2] Mykhaylo Andriluka, Leonid Pishchulin, Peter Gehler, and Bernt Schiele. 2014. 2D Human Pose Estimation: New Benchmark and State of the Art Analysis. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3686–3693.

[3] Rik J. Bouwmeester, Federico Paredes-Vallés, and Guido C. H. E. de Croon. 2023. NanoFlowNet: Real-time Dense Optical Flow on a Nano Quadcopter. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 1996–2003.

[4] Lukas Cavigelli, Philippe Degen, and Luca Benini. 2017. CBinfer: Change-Based Inference for Convolutional Neural Networks on Video Data. In *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*. 1–8.

[5] Tatjana Chavdarova, Pierre Baqué, Stéphane Bouquet, Andrii Maksai, Cijo Jose, Timur M. Bagautdinov, Louis Lettry, Pascal Fua, Luc Van Gool, and François Fleuret. 2018. WILDTRACK: A Multi-Camera HD Dataset for Dense Unscripted Pedestrian Detection. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 5030–5039. doi:10.1109/CVPR.2018.00528

[6] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. 2019. MMDetection: Open MMLab Detection Toolbox and Benchmark. *CoRR* abs/1906.07155 (2019).

[7] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR* abs/1410.0759 (2014).

[8] Christopher B. Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[9] David B. D'Ambrosio, Saminda Abeyruwan, Laura Graesser, Atil Iscen, Heni Ben Amor, Alex Bewley, Barney J. Reed, Krista Reymann, Leila Takayama, Yuval Tassa, Krzysztof Choromanski, Erwin Coumans, Deepali Jain, Navdeep Jaitly, Natasha Jaques, Satoshi Kataoka, Yuheng Kuang, Nevena Lazic, Reza Mahjourian, Sherry Moore, Kenneth Oslund, Anish Shankar, Vikas Sindhwani, Vincent Vanhoucke, Grace Vesom, Peng Xu, and Pannag R. Sanketi. 2025. Achieving Human Level Competitive Robot Table Tennis. In *IEEE International Conference on Robotics and Automation, ICRA 2025, Atlanta, GA, USA, May 19-23, 2025*. IEEE, 74–82. doi:10.1109/ICRA55743.2025.11127501

[10] Sankeerth Durvasula, Yushi Guan, and Nandita Vijaykumar. 2023. Ev-Conv: Fast CNN Inference on Event Camera Inputs for High-Speed Robot Perception. *IEEE Robotics Autom. Lett.* 8, 6 (2023), 3174–3181.

[11] Davide Falanga, Philipp Foehn, Peng Lu, and Davide Scaramuzza. 2018. PAMPC: Perception-Aware Model Predictive Control for Quadrotors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 1–8. doi:10.1109/IROS.2018.8593739

[12] Davide Falanga, Suseong Kim, and Davide Scaramuzza. 2019. How Fast Is Too Fast? The Role of Perception Latency in High-Speed Sense and Avoid. *IEEE Robotics Autom. Lett.* 4, 2 (2019), 1884–1891. doi:10.1109/LRA.2019.2898117

[13] Davide Falanga, Kevin Kleber, and Davide Scaramuzza. 2020. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci. Robotics* 5, 40 (2020), 9712. doi:10.1126/SCIROBOTICS.AAZ9712

[14] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 253–267. doi:10.1145/3620666.3651378

[15] Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry P. Vetrov, and Ruslan Salakhutdinov. 2017. Spatially Adaptive Computation Time for Residual Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 1790–1799.

[16] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2023. MegaBlocks: Efficient Sparse Training with Mixture-of-Experts. In *Proceedings of the Sixth Conference on Machine Learning and Systems, MLSys 2023, Miami, FL, USA, June 4-8, 2023*, Dawn Song, Michael Carbin, and Tianqi Chen (Eds.). mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2023/hash/5a54f79333768effe7e8927bcccffe40-Abstract-mlsys2023.html

[17] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

[18] Yizhao Gao, Baoheng Zhang, Xiaojuan Qi, and Hayden Kwok-Hay So. 2023. DPACS: Hardware Accelerated Dynamic Neural Network Pruning through Algorithm-Architecture Co-design. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 237–251. doi:10.1145/3575693.3575728

[19] Benjamin Graham, Martin Engelcke, and Laurens van der Maaten. 2018. 3D Semantic Segmentation with Submanifold Sparse Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*

*(CVPR)*.

[20] Amirhossein Habibian, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. 2021. Skip-Convolutions for Efficient Video Processing. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2695–2704.

[21] Drew Hanover, Antonio Loquercio, Leonard Bauersfeld, Angel Romero, Robert Penicka, Yunlong Song, Giovanni Cioffi, Elia Kaufmann, and Davide Scaramuzza. 2024. Autonomous Drone Racing: A Survey. *IEEE Trans. Robotics* 40 (2024), 3044–3067. doi:10.1109/TRO.2024.3400838

[22] En-Ming Huang, An-Cheng Chang, Bai-Cheng Jeng, Shih-Hao Hung, and H. T. Kung. 2026. Decoding the Skew: Distribution-Aware MoE Inference with Adaptive Kernel Dispatch. *CoRR* abs/2607.23099 (2026). arXiv:2607.23099 doi:10.48550/ARXIV.2607.23099

[23] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. 2023. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

[24] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman P. Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (2017).

[25] Fanrong Li, Gang Li, Xiangyu He, and Jian Cheng. 2021. Dynamic Dual Gating Neural Networks. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*. 5310–5319.

[26] Shih-Chieh Lin, Yunqi Zhang, Chang-Hong Hsu, Matt Skach, Md. Enamul Haque, Lingjia Tang, and Jason Mars. 2018. The Architectural Implications of Autonomous Driving: Constraints and Acceleration. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2018, Williamsburg, VA, USA, March 24-28, 2018*, Xipeng Shen, James Tuck, Ricardo Bianchini, and Vivek Sarkar (Eds.). ACM, 751–766. doi:10.1145/3173162.3173191

[27] Renyuan Liu, Yuyang Leng, Shilei Tian, Shaohan Hu, Chun-Fu (Richard) Chen, and Shuochao Yao. 2024. DynaSpa: Exploiting Spatial Sparsity for Efficient Dynamic DNN Inference on Devices. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems, SenSys 2024, Hangzhou, China, November 4-7, 2024*. ACM, 422–435. doi:10.1145/3666025.3699348

[28] Nico Messikommer, Daniel Gehrig, Antonio Loquercio, and Davide Scaramuzza. 2020. Event-Based Asynchronous Sparse Convolutional Networks. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 415–431.

[29] Asit K. Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating Sparse Deep Neural Networks. *CoRR* abs/2104.08378 (2021).

[30] NVIDIA. 2026. Parallel Thread Execution ISA Version 9.5. https://docs.nvidia.com/cuda/parallel-thread-execution/

[31] NVIDIA. 2026. TensorRT: Working with Dynamic Shapes. https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/work-dynamic-shapes.html

[32] Federico Paredes-Vallés and Guido C. H. E. de Croon. 2021. Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3445–3454.

[33] Mathias Parger, Chengcheng Tang, Thomas Neff, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2023. MotionDeltaCNN: Sparse CNN Inference of Frame Differences in Moving Camera Videos with Spherical Buffers and Padded Convolutions. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. 17246–17255.

[34] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2022. DeltaCNN: End-to-End CNN Inference of Sparse Frame Differences in Videos. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 12487–12496.

[35] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David R. Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 683–698. doi:10.1145/3620665.3640426

[36] Mengye Ren, Andrei Pokrovsky, Bin Yang, and Raquel Urtasun. 2018. SBNet: Sparse Blocks Network for Fast Inference. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 8711–8720.

[37] Harisankar Sadasivan, Muhammed Emin Ozturk, Muhammad Osama, Chris Millette, Astha Rai, Maksim Podkorytov, John Afaganis, Carlus Huang, Jing Zhang, and Jun Liu. 2025. Stream-K++: Adaptive GPU GEMM Kernel Selection and Scheduling for AI Using Bloom Filters. In *High Performance Computing - ISC High Performance 2025 International Workshops, Hamburg, Germany, June 10-13, 2025, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 16091)*, Sarah Neuwirth, Arnab Kumar Paul, Tobias Weinzierl, and Erin Claire Carson (Eds.). Springer, 480–493. doi:10.1007/978-3-032-07612-0_37

[38] Haichen Shen, Jared Roesch, Zhi Chen, Wei Chen, Yong Wu, Mu Li, Vin Sharma, Zachary Tatlock, and Yida Wang. 2021. Nimble: Efficiently Compiling Dynamic Neural Networks for Model Inference. In *Proceedings of the Fourth Conference on Machine Learning and Systems, MLSys 2021, virtual, April 5-9, 2021*, Alex Smola, Alex Dimakis, and Ion Stoica (Eds.). mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2021/hash/5b47430e24a5a1f9fe21f0e8eb814131-Abstract.html

[39] Spconv Contributors. 2022. Spconv: Spatially Sparse Convolution Library. https://github.com/traveller59/spconv.

[40] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 13438–13444.

[41] Shawn Tan, Yikang Shen, Rameswar Panda, and Aaron C. Courville. 2024. Scattered Mixture-of-Experts Implementation. *CoRR* abs/2403.08245 (2024). arXiv:2403.08245 doi:10.48550/ARXIV.2403.08245

[42] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *Proceedings of Machine Learning and Systems (MLSys)*.

[43] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 225–239.

[44] Thomas Verelst and Tinne Tuytelaars. 2020. Dynamic Convolutions: Exploiting Spatial Sparsity for Faster Inference. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2320–2329.

[45] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martin Martin, Cewu Lu, Li Fei-Fei, and Silvio Savarese. 2019. DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3343–3352.

[46] Kunyun Wang, Shuo Yang, Jieru Zhao, Wenchao Ding, Quan Chen, Jingwen Leng, and Minyi Guo. 2025. SparseTem: Boosting the Efficiency of CNN-Based Video Encoders by Exploiting Temporal Continuity. In *Advanced Parallel Processing Technologies - 16th International Symposium, APPT 2025, Athens, Greece, July 13-16, 2025, Proceedings*. 246–256.

[47] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. 2019. FastDepth: Fast Monocular Depth Estimation on Embedded Systems. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 6101–6108.

[48] Zhenda Xie, Zheng Zhang, Xizhou Zhu, Gao Huang, and Stephen Lin. 2020. Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 531–548.

[49] Jiacheng Yang, Christina Giannoula, Jun Wu, Mostafa Elhoushi, James Gleeson, and Gennady Pekhimenko. 2024. Minuet: Accelerating 3D Sparse Convolutions on GPUs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. 786–802.

[50] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[51] Changqian Yu, Jingbo Wang, Chao Peng, Changxin Gao, Gang Yu, and Nong Sang. 2018. BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 334–349.

[52] Bojian Zheng, Ziheng Jiang, Cody Hao Yu, Haichen Shen, Joshua Fromm, Yizhi Liu, Yida Wang, Luis Ceze, Tianqi Chen, and Gennady Pekhimenko. 2022. DietCode: Automatic Optimization for Dynamic Tensor Programs. In *Proceedings of the Fifth Conference on Machine Learning and Systems, MLSys 2022, Santa Clara, CA, USA, August 29 - September 1, 2022*, Diana Marculescu, Yuejie Chi, and Carole-Jean Wu (Eds.). mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2022/hash/f89b79c9a28d4cae22ef9e557d9fa191-Abstract.html

[53] Alex Zihao Zhu, Dinesh Thakur, Tolga Özaslan, Bernd Pfrommer, Vijay Kumar, and Kostas Daniilidis. 2018. The Multi Vehicle Stereo Event Camera Dataset: An Event Camera Dataset for 3D Perception. *CoRR* abs/1801.10202 (2018).